Ability

Ability.cs

Abstract class for abilities. Contains variables and method necessary for cooldown systems.
Charge represents a use of an ability.
Charge point represents the amount of progress to complete a recharge, for example, if it has 84 charge points out of 100 charge points necessary to complete a recharge, this means recharge is 84% complete. The amount of charge point necessary to recharge and the rate at which the ability gets charge points can both be customized.

To use the ability script, create a new script which inherits from Ability
Set the abilityStat variable to a reference of an ability stat scriptable object for this custom ability. Make sure to set its variables in the ability stat to the desired values
Override Startup and Cleanup methods.

- Startup method is called at the end of the initialize method and it is designed to contain custom code for the ability that needs to be activated on awake, such as setting all necessary variables.
- Cleanup method is called by the ability manager when it is removed from the ability manager.

Create a method which is called when the ability activates.
This method needs to have a parameter of a type struct designed to be used as a parameter for an event.
This method needs to be subscribed to event bus with matching type for the parameter in the Startup method
This method needs to be unsubscribed to the event bus in the Cleanup method
This setup allows for the ability to be activated when the event occurs.

ConsumeCharge()

Uses a charge of the ability.
Parameter: int ChargeConsumed
Result: If current charge is above 0, it will subtract current charge by charge consumed. Does not go below 0

CanActivate()

Returns a bool based on if the ability is meant to be able to be activated.
Parameter:n/a

Result:Return True if ability can be activated, return False if it cannot be activated

ActivateReload()

Sets recharge in progress to true if there are missing charges.
Parameter: n/a
Result: Set recharge in progress to true if there are missing charges, this allows the ability to recharge

RecoverChargePoint()

Increases charge point by the amount in charge added parameter
Parameter: Float chargeAdded
Result: If recharge in progress is true, it increases current charge points by the amount specified by charge added. If the amount added is greater than charge necessary to complete a reload, the extra charge point overflows into the next charge point progression.

ReloadOverTime()

Recovers charge point based on amount of time passed
Parameter:float timeElapsed
Result:Recharges charge point by its recharge rate based on time passed.

InterruptReload()

Resets recharge progress
Parameter:n/a
Result:Recharge progress is set to 0

GetCurrentCharge()

Returns the value of current charge

GetChargePointProgress()

Returns the value of charge point progress

ActiveAbility

A child class of Ability that is designed to be used for abilities that activate from input.

SetUpInput()

Connects the ability to the current input. Make sure that it has an associated input unit that is different from the other abilities that the character uses. If there is already an ability with a set input unit, the ability will not be available.
Parameter: n/a
Result: the ability can now be used by the input.

AbilityManager.cs

Stores all abilities that the character has in a list. Necessary for the ability to function as intended. Add this script to the player game object in the inspector.

AddAbility()

Adds ability to the list and its associated UI.
This method is normally called when the game object with the ability script is called
Input: ability's game object
Result: ability is added to its ability list and adds the ability's UI on the canvas

CanUseAbility()

Return bool based on if use of an ability is possible based on if the ability can interrupt others and if there is a different ability in use
Input: ability to be activated
Result: return true if ability can be used, false otherwise

NotifyAbilityStarted()

Sets the ability to be in use. Intended to be called when the ability is used
Input: ability to be activated
Result: sets current ability to input

NotifyAbilityEnded()

Sets the ability to not be in use. Intended to be called when the ability ends. If input is same as ability in use, then the ability ends
Input : ability that ended
Result: current ability set to null